Release cadence improves a score but misses the review queue

James N. Jean IV
Independent research and implementation study 30 October 2026
Reproducible candidate, not peer reviewed

Decision brief

A dependency owner may need time to inspect a project's release process before an upgrade deadline. This study tests whether three past stable-release intervals help identify a subsequent ninety-day gap in stable GitHub releases. The outcome is a recorded publishing pattern; actual maintenance activity is unobserved.

The frozen cadence rule improves final macro Brier score from 0.286162 to 0.249727 on 39 origins from four other projects. Its predeclared probability-at-least-one-half review rule flags none of the 10 observed gaps. The probabilistic comparison improves, while the review output fails to surface any positive case.

Final outcome	Pooled baseline	Frozen cadence rule
Macro Brier, lower is better	0.286162	0.249727
Micro Brier, lower is better	0.223042	0.191728
Macro log loss, nats	0.903990	0.809713
Observed gaps flagged	0/10	0/10
Observed gaps missed	10	10

Practical recommendation. Keep cadence as contextual evidence in a dependency review, with the release channel and actual maintenance evidence attached. The tested threshold produces an empty queue and should not serve as an automatic routing policy. A prospective evaluation should measure the decisions and outcomes a dependency owner actually cares about.

Evidence boundary. Eight familiar Python-tooling repositories form an authored cohort in a current public snapshot. Edited or deleted releases prevent an original-vintage reconstruction. Origins overlap, and projects share organizations and an ecosystem. There is no representative sample or independent maintenance-status label. Deployment benefit, abandonment and security outcomes are unmeasured.

Usable output. Saved minimal metadata, a fixed model, all candidates, independent checks and an offline exercise support a contextual review item without contacting a maintainer.

1. What the source actually measures

GitHub's releases endpoint lists published release records; ordinary Git tags without a release use a separate endpoint. Stable here means captured draft and prerelease flags are both false. Projects can publish packages or continue development through other channels. The target concerns stable GitHub release cadence only.

The snapshot contains 484 release entries from 18 preserved response entities totaling 2,248,070 decoded bytes: one initial public rate-limit response, eight repository metadata responses and nine release-list pages. Acquisition follows publisher pagination to its actual end. All repositories are public non-forks in current metadata.

Repository	Group	Source	Train	Valid.	Final
python-attrs/attrs	Development	27	9	1	0
encode/httpx	Final	51	0	0	5
pallets/click	Development	30	9	4	0
psf/black	Final	57	0	0	11
pallets/flask	Development	38	12	8	0
psf/requests	Final	19	0	0	6
Textualize/rich	Development	177	138	17	0
pytest-dev/pytest	Final	85	0	0	17

The assignment sorts SHA256 values of lowercase repository slugs and alternates development and final groups before acquisition. Distinct IDs do not establish independent projects. Source counts include excluded entries and dates outside the origin windows. Every qualification reason is retained.

Complete responses remain private. Public files contain IDs, timestamps, state flags, hashes and new calculations, excluding release prose, profiles, avatars and assets. Local hashes describe Requests-decoded entities rather than publisher checksums. The original GH Archive proposal was narrowed before acquisition because its software/documentation license does not blanket-license archived third-party event content. Contributor concentration and bot attribution are not executed analyses.

The first collector stopped at a numeric repository-ID next link rejected by its initial named-path guard. Fifteen successful originals and the code are preserved. A correction accepts the exact verified ID while retaining origin/page restrictions. One continuation reused all prior bodies after hash checks and made three new requests. Six pagination fixture methods cover the correction. No denied source or outcome-dependent cohort change was involved.

2. A fixed model with a retained search

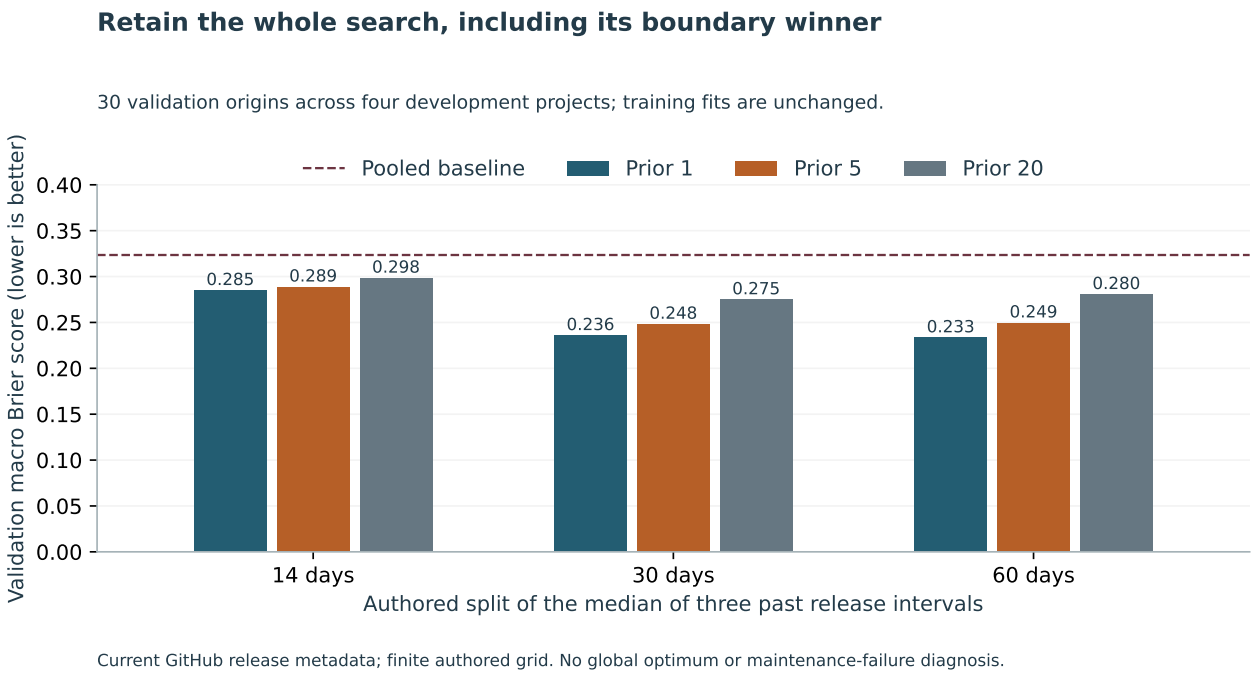
At each eligible origin, the predictor is the median of the three most recent elapsed release intervals, including the interval ending at that origin. Events sort by publication time then release ID; a same-time later release ends the next interval at zero. Missing timestamps, insufficient history and incomplete windows retain separate exclusion reasons.

Training uses development-project origins in 2020–2022 with ninety-day windows ending by the training boundary. Validation uses those projects' 2023 origins with windows ending by its boundary. The target is $y = 1$ when no next stable release occurs within ninety elapsed days; an event exactly at the horizon counts as within it. Training has 168 origins and 12 gaps; validation has 30 origins and 4 gaps.

For training count n and positives k, the authored pooled baseline is $p_0 = (k + 1)/(n + 2)$. Candidate cutpoints are 14, 30 and 60 days; prior strengths are 1, 5 and 20. Each stratum uses

$$p_s = \frac{k_s + ap_0}{n_s + a}.$$

Empty strata return p_0. All 9 authored hypotheses and runtimes are retained. They imply no global-optimum claim.



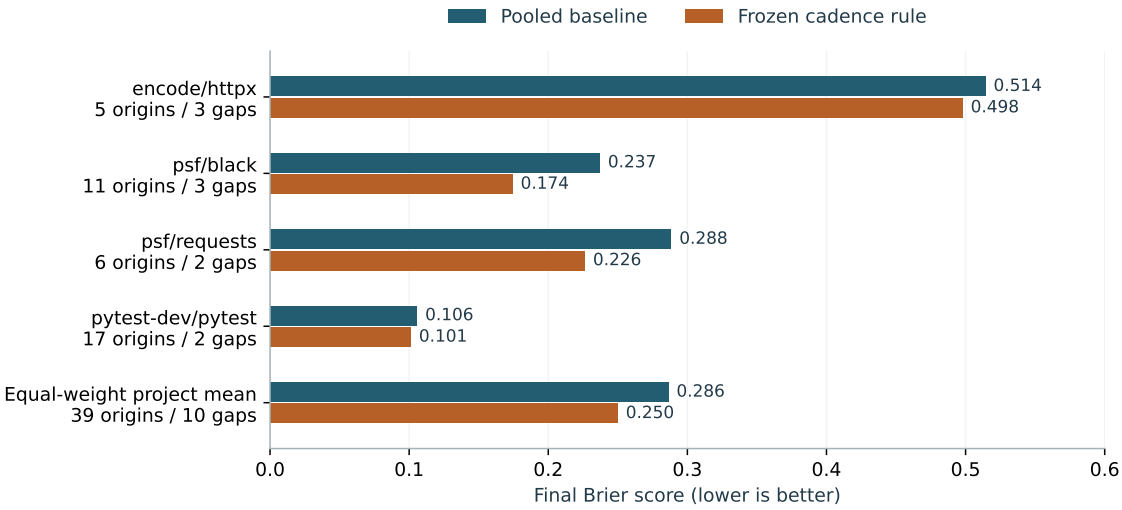
Selection minimizes validation Brier averaged equally across projects; exact ties favor a larger prior then smaller cutpoint. The selected 60-day split and prior 1 are boundary winners. Validation macro Brier is 0.233250 versus 0.323495. There is no validation refit. A committed manifest of 32 dependencies precedes final feature, label and score extraction. Source envelopes already contain final metadata, so collection was not blinded.

3. Other projects and later origins

Final evaluation applies the unchanged training fit to the other four projects' 2024–2025 origins, with full windows ending by the final boundary. There are 39 origins and 10 gaps. No final outcome chooses a new cutpoint, prior, horizon or threshold.

Probability scores improve on this small final cohort

Four other projects, 2024–2025 origins; current snapshot, not historical vintages.



The 0.5 review rule still misses all ten observed gaps. Lower Brier is not measured operational benefit.

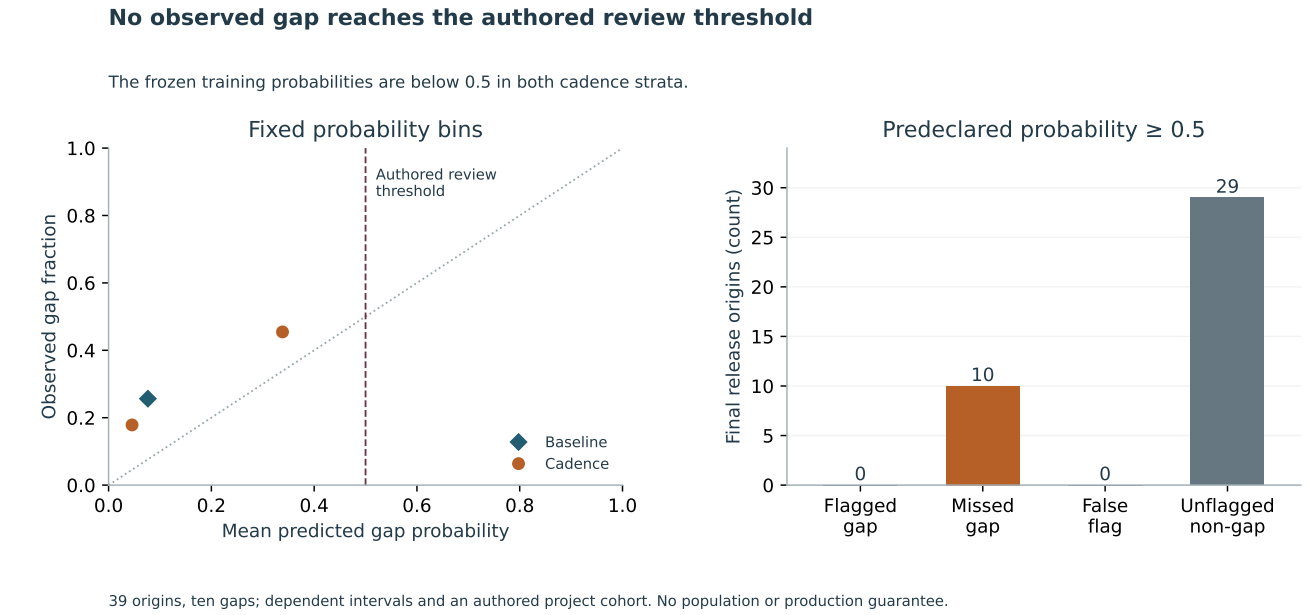
Macro Brier weights projects equally; micro Brier weights origins equally. Both improve on this finite cohort. Each project's Brier is lower for the cadence rule. Other scores can worsen: encode/httpx log loss rises from 1.574331 to 1.616603 nats in a subgroup with five origins and three gaps.

The training baseline probability is 0.076471. The selected shorter-cadence stratum has 154 training origins and 7 gaps, while its longer-cadence stratum has 14 origins and 5 gaps. The resulting probabilities are 0.045655 and 0.338431. Both remain below the authored review threshold.

Separate IDs and later dates preserve the declared split while leaving ecosystem dependence and current snapshot omissions unresolved. A difference in cadence does not identify a causal effect of contributor behavior or dependency policy. Actual outages, avoided costs, workload reductions and security outcomes are outside the observed data.

4. The review failure is part of the result

At $p \geq 0.5$, final counts are true positives 0, false positives 0, false negatives 10 and true negatives 29. Precision is undefined because no flags are emitted. Recall is zero of 10 observed gaps. The baseline also emits no flags. The threshold remains unchanged after final inspection.



Fixed probability bins show underprediction. The selected shorter-cadence bin contains 28 origins with five gaps; the longer-cadence bin contains eleven origins with five gaps. Both frequencies exceed their frozen probabilities. Lower aggregate Brier is not a calibration certificate; Brier and log loss reflect more than calibration alone.

Separate descriptive product-limit summaries use observed inter-release durations and right-censoring at each phase boundary. At ninety days, full-window inclusion makes the final empirical remaining-gap fraction ten of thirty-nine. Overlapping origins and current source omissions prevent an independent-sampling interpretation or project-level survival diagnosis.

The offline exercise accepts three authored integer day intervals and returns an estimate with its scope. It makes no requests, sends no notification and replaces no dependency. A useful next study would prospectively record release channels, review decisions and actual maintenance outcomes across a broader cohort before selecting an operational policy.

5. Reproducibility and checks

DuckDB JSON and SQL reconcile exact private response entities to minimal public source fields in 6,024 conditions. A separate SQLite window implementation rebuilds chronology. Fraction arithmetic checks probabilities, Brier and denominators; 75-digit Decimal log-loss comparisons use tolerance 2×10^{-12}. Development checks cover 8,970 conditions, final checks 2,272. These finite checks do not establish universal reliability.

21 fixture methods cover chronology, horizon boundaries, same-time ordering, missing/timezone data, source flags, empty strata, undefined denominators, log flooring, selection ties, risk sets and pagination identity. Initial measured runtimes are retained; replay writes fresh runtime receipts separately. Current-checkout development replays ran under CPython 3.10.11 and 3.12.14.

```
python -m unittest discover -s tests -v
python src/freeze_proof.py
python src/replay.py --output ../fresh-replay
python src/review_cli.py examples/authored-intervals.json
python src/verify_figures.py
```

Use a fresh output outside the checkout. Captured-source replay makes no new network requests. Runtime is provenance rather than expected replay bytes. Public replay checks saved minimal facts and source proof; private originals remain excluded. Release requires an isolated full-history replay, source/figure hashes, actual SVG geometry, every rendered paper page, a scoped publication audit and exact GitHub CI/tree/tag evidence. Check the chosen revision's evidence when using this package.

Primary sources and method references

GitHub. *REST API endpoints for releases*; pagination and rate-limit documentation. https://docs.github.com/en/rest/releases/releases. Published release records differ from ordinary Git tags.

GH Archive. *Licenses*. https://github.com/igrigorik/gharchive.org#licenses. Software and website rights differ from third-party event rights.

NIST/SEMATECH. *Empirical model fitting: Kaplan–Meier approach*. https://www.itl.nist.gov/div898/handbook/apr/section2/apr215.htm. Product-limit summaries here are descriptive.

scikit-learn developers. *Probability calibration*. https://scikit-learn.org/stable/modules/calibration.html. Brier and log loss assess more than calibration alone. The small model is implemented directly, without a scikit-learn dependency.

Publication scope. This independent manuscript is eligible for the reviewed program subset. Separate formal-publication projects, including Azure optimization, remain private. Third-party prose, profiles and manuscripts are excluded.